From CV-JD Pairs to Learning Paths: Skill-Gap Driven Course Recommendation

*

Abstract. In Human Capital Management, candidates frequently struggle to identify their missing skills or to align their competencies with job requirements, creating uncertainty in both job applications and educational choices. To bridge the gap between a candidate's existing skills and an employer's expected skills, we introduce a new task: course recommendation for Curriculum Vitae – Job Description (CV–JD) pairs. We propose a pipeline in four stages. First, a pre-trained LLM extracts skills and their proficiency levels from the CV and the JD, from which we estimate the per-skill gap. Second, we organize the course catalog as a hierarchical tree to support efficient retrieval. Third, candidate courses are retrieved deterministically according to the computed skill gap. Fourth, the retrieved courses are ranked by their relevance to the gap to form a level-progressive learning path. To support this task, we annotate a course-recommendation dataset for the Information Technology field, comprising 5,856 courses and 36 labeled CV–JD pairs. Experiments show that the framework produces well-suited learning paths that guide candidates from their existing skills toward the levels employers expect, while confining the LLM to extraction keeps the retrieval and ranking stages reproducible.

1 Introduction

Human Capital Management (HCM) enables organizations to build value for themselves and their employees through tasks such as talent acquisition, succession planning, employee development, and workforce agility. Much of the information driving these HCM tasks exists as unstructured natural language text, including curriculum vitae (CVs), job descriptions (JDs), performance reviews, and learning records. Consequently, Natural Language Processing (NLP) has become an emerging technology [10] for: (i) turning free-form text into structured representations of skills, roles, and requirements, for downstream tasks, such as screening CVs and matching candidates to jobs [2]; (ii) personalizing training programs [1]; and (iii) predicting employee turnover [11].

* Corresponding author:

As labour markets grow more dynamic and the demand for specialized competencies accelerates, particularly in the Information Technology (IT) sector, reasoning over textual human capital data at scale has become a competitive necessity. In parallel, the proliferation of online learning platforms has motivated educational recommendation systems that guide learners toward relevant courses [1][6], supporting workforce up-skilling and re-skilling. These applications collectively aim to reduce friction between the supply of talent and the demand expressed by employers. Recent advances [3][8][15][16] have improved the quality of these recommendation systems. These capabilities have raised the ceiling on tasks such as skill extraction, semantic job matching, and profile summarization, and have made it feasible to reason about competencies with semantic rather than merely match surface strings.

Despite this progress, existing work in HCM leaves a critical gap. Most systems are oriented toward matching whether a candidate is suitable for a role, but stop short of supporting the candidate who is found to be "unsuitable." When a mismatch is detected, current tools rarely explain which specific competencies are missing, at what proficiency level, or how the candidate might acquire them. Lexical matchers compound this shortfall through semantic blindness: they misinterpret disjunctive requirements. For an example, matchers treat `''C# or Java''` as two mandatory skills rather than interchangeable alternatives and disregard proficiency levels entirely, and therefore a beginner's exposure to a skill is indistinguishable from an expert's mastery.

To address this gap, we approach a bridge task of *course recommendation for CV–JD pairs*. Specifically, this task involves a level-aware and disjunction-aware formulation of CV–JD skill gaps, combined with reproducible deterministic course retrieval and ranking. Rather than merely judging fit, the task connects a candidate's existing skills, extracted from the CV, to the expected skills, demanded by a target JD. Then, this task recommends a learning path that closes the intervening gap. This bridge task reformulate the problem from binary matching to constructive guidance: the system must (i) identify the skills and proficiency levels present in the CV and required by the JD, (ii) quantify the per-skill gap between them, and (iii) retrieve and order courses into a progressive path that advances the candidate from their current competencies toward the employer's expectations. To the best of our knowledge, the formulation from CV-JD to learning path has not been systematically studied in prior HCM research.

The main contributions of this paper are as follows:

1) *A bridge task and dataset.* We formulate course recommendation for CV–JD pairs as a bridge between recruitment demand and educational supply. Then, we annotate a dataset in the IT domain comprising 5,856 courses and 36 labeled CV–JD pairs to enable its evaluation.
2) *A Hybrid Recommendation Framework.* We proposed a framework of four stages to recommend courses for candidates. First, skills and levels are extracted from CVs and JDs by pre-trained Large Language Models. Second, skill gap between existing skills and expected skills is calculate. Third, based

on the gap, candidate courses are retrieved using a pre-built hierarchical course tree. Forth, the candidate courses are ranked by the relevant score of skill, level and redundancy.

3) *Empirical validation.* We evaluate each component of the pipeline, isolating where the LLM genuinely adds value and where deterministic computation is sufficient. **..... (se viet tiep sau khi doc ket qua thuc nghiem)**

2 Related Work

The research most closely related to this work lies in the domains of learning path recommendation and course recommendation systems. Two comprehensive surveys [8][15] provide an overview of learning path recommendation, categorizing existing approaches according to their data sources, modeling techniques, and application domains. Among the approaches, knowledge graph based solutions [4] account for the majority.

A central challenge in learning path recommendation is the construction of knowledge structures that capture the dependencies and relationships among learning items. Zhou et al. [14] propose a method based on formal concept analysis to derive a concept lattice from historical learning data, which they then use to generate personalized learning paths. Meng et al. [7] describe a platform that integrates data from learning management systems, social networks, and learner portfolios to create a heterogeneous information network as the basis for recommendation.

Course recommendation systems have incorporated explanatory mechanisms to improve the explainability of their suggestions [6]. The rise of deep learning has motivated the application of sequence modeling techniques to learn complex patterns in learner behavior. Lim et al. [5] propose a goal-driven approach that uses a recurrent neural network to model the learner's knowledge state and a modified beam search to generate learning paths toward a specified goal. Yang et al. [13] extend this paradigm by incorporating the forgetting curve to capture the temporal dynamics of knowledge retention and adapt the learning path accordingly.

While most methods rely on supervised learning and explicit rating data, several recent works explore unsupervised and generative approaches. Wei et al. [12] propose a transformer-based framework that masks the historical path to infer a learner's next action, which enables effective learning path generation even in cold-start settings. Nasrin et al. [9] apply multi-task learning to jointly model course selection and completion, demonstrating improved recommendation quality by capturing the interdependence between these tasks.

Despite these advances, most methods share a common assumption: the learner's goal either known a prior or can be inferred from their historical behavior. In practice, however, learners often struggle to articulate their goals precisely, and their past actions may not fully reflect their aspirations. To fill this gap, some works extract skills from a CV, compare them with a selected vacancy, identify missing skills, and recommend courses providing those skills [3][16].

Our work addresses this gap by introducing a new formulation of the learning path recommendation problem: given a learner's current competencies and a target job description, the task is to generate a sequence of courses that bridges the two. By grounding the goal in a specific target role, this framing makes the learning objective explicit and aligns the recommended path with labour market demand.

3 The Hybrid Recommendation Framework

There are two main phases: offline phrase for constructing a hierarchical course tree and online phase for skill gap driven course recommendation.

3.1 Offline Phase: Constructing the Hierarchical Course Tree

To avoid the high computational cost of searching for thousands of courses at the request time, we prepare in advance a hierarchical course tree. From courses crawled from Coursera, skill variants are normalized to one canonical name, e.g., unifying terms `react.js` as `ReactJS`, removing tool names and vague phrases. Then, courses are classified into a pre-defined hierarchical categories. For experiment in this paper, we decide 11 categories and Fig. 1 illustrates a piece of hierarchical course tree in IT domain, in which the course names are in blue and the hierarchical category is in black. This hierarchical course tree is reviewed and revised manually to ensure the quality.

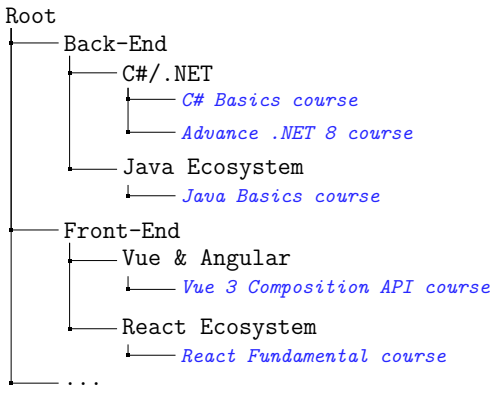


Fig. 1. A piece of hierarchical course tree: category is in black and courses are in blue.

3.2 Online Phase: Skill-Gap-Driven Course Recommendation

The online phase for course recommendation based on the skill-gap includes four main steps: 1) extracting skills and levels from a given CV-JD pair; 2) computing

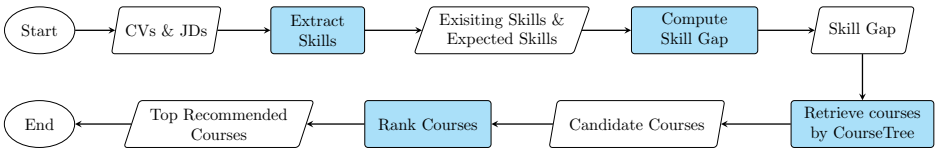


Fig. 2. Overview pipeline for course recommendation system.

the skill gap between existing skills and expected skills; 3) retrieving candidate courses via the course tree; and finally 4) ranking them into a Top-K learning path. Fig. 2 shows the overall pipeline of this phrase.

3.2.1 Extracting Skills and Levels

Leveraging recent advancements of pretrained large language models (LLMs), we employ them to carry out the task of parsing unstructured text from CVs and JDs for skills and corresponding levels. First, text is normalized by unifying choice separators and escaping braces, then truncated to stay within the input context budget of the LLM, such as Qwen3-Coder-480B. Second, we send the text to the cloud-hosted LLM, constraining it to return skills as *pairs* of `(skill, level)`, where `skill` is the skill name and `level` is the proficiency indicated by a value in $\{1 : \text{familiar}, 2 : \text{experienced}, 3 : \text{proficient}\}$. Note that, LLM itself never output zero, therefore, in case a job-required skill is unavailable from the candidate skill set, a value 0 is paired with the required skill.

The exact template used for skill extraction is as following prompting, in which we omit alternative-skill clause for the case of CVs since candidates do not mention alternative skills:

```
    [SYSTEM ROLE] You are a core Skill Extraction Engine. Task: extract hard technical
skills from the job description.
    Output MUST be a JSON array: [{"skill_name": string, "level": 1-3}]
    [STRICT RULES]
    1. Skill scope (maximally inclusive): languages (C#, Java), frameworks (Angular,
ExpressJS), databases (Oracle, MongoDB), DevOps/cloud (Docker, Kubernetes, CI/CD),
version control (Git); technical concepts (SSR, CSR, ETL, Microservices, Security, JWT,
Caching).
       BLACKLIST (never extract): soft skills (Teamwork, Communication,
Problem-solving); language proficiency (English); business/domain knowledge (Banking,
Finance, E-commerce); overly generic terms (Backend, Frontend, Database, DevOps, IT).
       If the JD lists mutually exclusive alternatives (e.g., ‘‘proficient in one of
Java, Node.js, or Python’’), extract each technology as a separate entry - do not merge
them - but cap the level at 2 (Standard) for every technology in the group, since only
one alternative is actually required.
       EXCEPTION (highest priority): if a technology is mentioned under a ‘‘Nice to have’’
/ ‘‘Preferred’’ heading, always assign level 1 (Basic), regardless of ‘‘or’’/list phrasing -
this overrides the alternative-group rule above.
    2. Level quantification (follow the exact wording): [3] Master - ‘‘proficient’’,
‘‘expert’’, ‘‘mastery’’, or ≥ 3 years. [2] Standard (default) - ‘‘experienced’’, ‘‘good working
knowledge’’, or 1-2 years. [1] Basic - ‘‘familiar’’, ‘‘basic certificate’’, or mentioned
without a clear proficiency level.
    [INPUT JD TO PROCESS]
    -- {jd_text} --
```

Extraneous text is handled by locating the outermost `[ / ]` before parsing. Each parsed record is then validated – non-empty `skill_name` and `level`∈

$\{1, 2, 3\}$ – and dropped rather than silently accepted if either check fails. Validated skills are resolved through a static alias map to one canonical name (e.g., *react* and *react.js* $\rightarrow$ *ReactJS*), enabling reliable comparison in Section 3.2.2 regardless of surface form.

3.2.2 Computing the Skill Gap

Algorithm 1 outlines the computation for what a job applicant is missing for the requirements of his desired job. Let S_j be the JD's required skills and S_u the candidate's skills; for skill s, $l_u(s)$ is the candidate's level (0 if absent) and $l_j(s)$ the required level.

Algorithm 1 Progressive Skill-Gap Computation

Require: Candidate skill levels S_u; job skill levels S_j
Ensure: *Gap*: missing skills with progressive levels
Gap $\leftarrow \emptyset$
for each skill $s \in S_j$ **do**
  $l_u \leftarrow \text{level}(s, S_u)$ {0 if s is absent from S_u}
  $l_j \leftarrow \text{level}(s, S_j)$
  if $l_u < l_j$ **then**
    $L_s \leftarrow [\max(1, l_u{+}1), \ldots, l_j]$
    Gap $\leftarrow$ *Gap* $\cup \{(s,\ L_s)\}$
  end if
end for
return *Gap*

Instead of a binary matching of met/unmet flag, the gap for a skill still below target is the ordered sequence of intermediate levels yet to be reached:

$$L_s = \big[\max(1,\ l_u(s) + 1),\ \ldots,\ l_j(s)\big]. \tag{1}$$

Intuitively, a gap is identified when a candidate having Java Spring Boot skill at Level 1 but the job requires Level 3, therefore the level gap in this case is $L_s = [2, 3]$. This phenomenon is overlooked when we use naive set-difference $S_j \setminus S_u$. Note that, our paper considers mandatory requirements from JD, the disjunctive requirements in form of *"nice to have"* or *"A or B"*) for preferable skills are discussed as future work.

3.2.3 Retrieving Candidate Courses via Hierarchical Course Tree

Given a skill gap G between a candidate and a job, this step finds candidate courses which possibly fill the missing skills. Keyword matching is semantically blind, while an LLM-routed retrieval that queries the model at every branch is too slow and, as Section 4.6 shows, no more accurate.

Deterministic skill-set pruning. Since each node has been annotated with the union of skills, descending into a branch is a simple intersection test: traverse

Algorithm 2 Deterministic DFS Tree Search with Skill-Set Pruning

Require: T: course tree in RAM, represented as a set of (name, node) pairs at each level, where each *node* is annotated with its aggregate skill set; G: candidate's skill gap
Ensure: *Candidates*: list of candidate courses

```
Stack ← ∅;  Candidates ← ∅
for each (name, node) ∈ T do
    Stack.push((name, node))
end for
while Stack ≠ ∅ do
    (path, node) ← Stack.pop()
    if G ∩ node.skills ≠ ∅ then
        {deterministic relevance test}
        if node is a leaf then
            Candidates.extend(node.courses)
        else
            for each (sub_name, sub_node) ∈ node do
                Stack.push((path + '→' + sub_name, sub_node))
            end for
        end if
    else
        prune: skip the entire branch {no model call, no cache}
    end if
end while
return Candidates
```

the branch if $G \cap \text{node.skills} \neq \emptyset$, otherwise prune the whole subtree without LLM model call, cache, or network dependency. An earlier LLM-routed design that cached per-node decisions was removed once this test made it redundant.

Traversal. Retrieval walks the tree with depth-first search using an explicit stack; each stack entry is a pair $(path, node)$, where *path* is the string of domain names traversed so far, e.g., *"Backend → Java Ecosystem"*. The *path* is then used to present the learning path by domain. The overall procedure of Algorithm 2 states the full procedure.

3.2.4 Ranking Courses

Retrieval yields a candidate pool; module D turns it into an ordered Top-K path, entirely deterministically.

Combo courses. Before ranking single-skill milestones, the system searches for *combo courses*. A *combo course* means a single course c from the candidate pool that is able to cover more than one gap at once. Coverage of a course c against the gap set G is:

$$E(c) = 0.5 \times \text{SkillMatch}(c) + 0.4 \times \text{LevelAlign}(c) - 0.1 \times \text{Redundancy}(c), \quad (2)$$

where $\text{SkillMatch}(c) \in \{0, 1\}$ indicates whether c genuinely teaches the target skill, which is verified by substring and alias matching; $\text{LevelAlign}(c) \in \{0, 0.5, 1\}$

rewards a level-appropriate course, gives partial credit to a foundational review below the target level, and zeroes out a course that overshoots it; and Redundancy$(c) \in [0, 1]$ is the fraction of c's skills that are irrelevant to the gap. A course is retained only if $E(c) \geq 0.7$; a course failing the skill match cannot reach threshold.

Deterministic ranking. For remaining milestones, one course per milestone is chosen by three signals in priority order: title match, gap coverage, and specificity (share of the course's skills relevant to the gap); the course ID breaks ties for full reproducibility. This replaces an earlier LLM-chunk ranking that was neither reproducible nor more accurate (Section 4).

4 Experiments and Evaluations

The evaluation

Rather than reporting one end-to-end accuracy figure, the evaluation is decomposed into five focused experiments (P1–P5), each isolating a single design decision: where the system relies on the LLM, and where it relies on deterministic computation. All metrics are computed by deterministic scripts against a manually labeled ground truth – the LLM is never used to grade its own output.

4.1 Data

Because extraction and recommendation require different ground truth, two labeled subsets were built. For retrieval and ranking (P3–P5), a balanced subset of 36 CV–JD pairs (12 per gap type missing-skill, missing-level, and disjunctive; pairs with no gap were excluded from this subset) was labeled at the course level: each course surfaced by the system, plus a random sample from the matching pool, was annotated *relevant* (1) or *not relevant* (0), yielding **595 labels**. For extraction (P1), the required skills of 36 job descriptions were independently annotated as a gold set of **131 skills**.

Labeling protocol. A fixed rule was applied to every label: the course title (or the JD's explicit wording) was the primary evidence for a relevance decision. An AI tool was permitted to assist analysis, but the final decision on every label was made by the author. A second independent annotator, enabling an inter-rater agreement score (e.g., Cohen's κ), is identified as future work rather than assumed.

4.2 Experimental Setup

The backend is implemented in Python with FastAPI; the course catalogue is stored in SQL Server and loaded once into an in-memory tree at start-up. The single LLM component (skill extraction) uses Qwen3-Coder-480B through the Ollama cloud endpoint, temperature fixed at 0, with JSON-schema-constrained prompting. All other components – tree search, gap filtering, ranking, and the

Table 1. Summary of the five experiments (P1–P5).

No.	Compares	Key results
P1	LLM vs. keyword extraction	F1 96.7% vs. 84.4%
P2	Filter vs. tree (tokens)	−99.8% (filter); tree +0.0%
P3	With/without $E(c)$	74.8% P; $E(c)$ ablation: 0.0 pp
P4	Deterministic vs. LLM pruning	99.2% vs. 63.3% recall, 25× faster
P5	Deterministic vs. LLM ranking	75.4% vs. 73.8% P; Jaccard 1.000 vs. 0.938

Table 2. Skill extraction: raw counts and Precision/Recall/F1 over 36 JDs (131 gold skills).

Method	Gold	Extracted	Correct	Precision	Recall	F1
Keyword baseline	131	132	111	84.1%	84.7%	84.4%
LLM (ours)	131	140	131	93.6%	100.0%	96.7%

evaluator $E(c)$ – run as deterministic Python with no network dependency, which is what makes the non-extraction stages reproducible.

Skill-set quality is measured with standard Precision, Recall, and F1 against human labels. Reproducibility is measured by running a deterministic stage multiple times on identical input and reporting the average pairwise Jaccard overlap of its outputs (1.0 = identical every run). Table 1 summarizes all five experiments and their headline result before each is discussed in detail.

4.3 Skill Extraction Accuracy (P1)

Question: is the LLM justified at the one stage where it is used? Extracted skill sets are compared against the 131-skill gold set, with a keyword baseline as control – a dictionary/regex-based matcher that scans the JD text for exact occurrences of known skill names (the same canonical skill list used elsewhere in the system), with no notion of proficiency level or disjunctive requirements.

Result. The LLM reached an F1-score of **96.7%** (Precision 93.6%, Recall 100.0%), against **84.4%** for the keyword baseline. The LLM correctly grouped all 10 disjunctive *"A or B"* requirements in the test set (10/10; keyword: 0/10) and correctly quantified proficiency levels for **83.2%** of skills, a task the keyword baseline cannot perform at all. This justifies confining the LLM to extraction: it is the only method able to resolve disjunction and levels from unstructured text.

4.4 Token Consumption Analysis (P2)

Question: does the hierarchical tree reduce the volume of data sent to the LLM? Token usage is compared across three configurations: a *naive flat scan*, which sends the full skill list of every course in the catalogue to the LLM with no prefiltering; a *deterministic filter alone*, which first discards courses with no skill overlap with the gap before sending the remainder; and the *filter combined with the hierarchical tree* (the proposed pipeline).

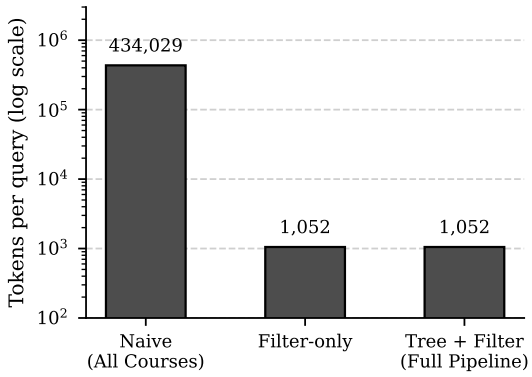


Fig. 3. Token consumption per query on a logarithmic scale.

Result. The naive scan averaged **434,029** tokens per query. The deterministic filter alone reduced this by **99.8%**, to **1,052** tokens. Adding the hierarchical tree on top of the filter contributed **0.0%** further reduction, since the tree returned the same candidate set as the filtered flat scan. The efficiency gain is therefore produced by the deterministic filter, not by the tree structure itself – the tree's role is organizational (Section 3.1), not a source of token savings.

4.5 Recommendation Quality and Guardrail Ablation (P3)

Question: how good is the end-to-end recommendation, and does the evaluator $E(c)$ earn its place? The full pipeline was run on all 36 labeled pairs and compared against the 595 human relevance labels; $E(c)$ was then ablated (removed) and the pipeline re-run.

Result. End-to-end, the system reached **74.8%** Precision, **61.0%** Recall, and **67.2%** F1 against human labels. Ablating $E(c)$ produced an identical result (0.0 percentage-point change): because every candidate had already passed deterministic skill-and-level filtering, all selected courses naturally cleared the $E(c) \geq 0.7$ threshold. $E(c)$ therefore acts as a safety net against upstream regressions rather than as an active filter on the current pipeline.

Question: how good is the end-to-end recommendation, and does the evaluator $E(c)$ earn its place? The full pipeline was run on all 36 labeled pairs and compared against the 595 human relevance labels; $E(c)$ was then ablated (removed) and the pipeline re-run.

4.6 Retrieval Pruning Strategy Ablation (P4)

Question: can the LLM be removed from course retrieval without losing recall? Three pruning strategies were compared: an LLM-by-name strategy (querying the model at each branch), the proposed deterministic skill-set pruning, and a hybrid of the two.

Table 3. Pruning strategy ablation (averages over 36 pairs).

Strategy	**Recall**	**LLM calls**	**Latency (s)**
LLM-by-name	63.3%	24.0	≈39
Deterministic (ours)	**99.2%**	**0**	**≈1.55**
Hybrid	99.2%	0.3	–

Table 4. Ranking strategy ablation: precision, reproducibility, and LLM cost.

Strategy	**Precision**	**Jaccard**	**LLM calls**
By-ID (control)	43.2%	1.000	0
LLM	73.8%	0.938	–
Smart-Det (ours)	**75.4%**	**1.000**	**0**

Result. Deterministic pruning reached **99.2%** recall in **≈1.55 s**, with **zero** LLM calls per query. The LLM-by-name strategy reached only **63.3%** recall, took **39 s**, and required **24** LLM calls per query – it occasionally pruned genuinely relevant branches. The hybrid strategy matched the deterministic strategy's recall with no measurable benefit. Deterministic pruning is therefore simultaneously more accurate, **≈25×** faster, and free of per-query model cost.

4.7 Ranking Strategy Ablation (P5)

Question: can the LLM be removed from course ranking as well? For each milestone's deterministic shortlist, one course was selected by three strategies: a By-ID baseline (lowest course ID, a determinism-only control), the previous LLM-based ranker, and the proposed deterministic ranker (Smart-Det), which scores by title match, gap coverage, and specificity. Precision was measured over the 36 pairs; reproducibility (pairwise Jaccard) was measured on a 6-task sample to bound LLM cost.

Result. Smart-Det reached the highest precision (**75.4%**), slightly above the LLM (**73.8%**), with perfect reproducibility (Jaccard = **1.000**) and zero LLM calls. The LLM was only **0.938** reproducible – under identical input and temperature 0, it still changed ≈6% of its selections between runs. The By-ID baseline is the decisive control: sorting only by course ID is just as reproducible (Jaccard = 1.000) but collapses precision to **43.2%**. This isolates the source of quality: determinism alone guarantees reproducibility, while the title/coverage/specificity scoring – not determinism per se – is what guarantees precision.

4.8 Threats to Validity

Four factors limit the external validity of these results. **(i)** The CV–JD test set is synthetic; results may differ on noisier real-world documents with misspellings or non-standard formatting. **(ii)** Ground truth was produced by a single annotator with AI-assisted analysis but a human final decision; a second annotator

and a Cohen's κ agreement score remain future work. **(iii)** Recall in P3 is estimated over a labeled sample (recommended items plus a random pool sample), not the full catalogue. **(iv)** The P5 reproducibility measurement uses a 6-task sample to control LLM cost; a larger sample is left for future validation. **(v)** The extraction model handles disjunctive requirements (e.g., *"proficient in one of Java, Node.js, or Python"*) heuristically: every technology inside such a group is extracted as a separate, individually required skill, with its stated proficiency level capped at Standard (2) rather than the group's original wording, to avoid overstating the requirement when only one alternative is actually needed. This is an approximation rather than a full disjunctive model (Section 5) – a JD requiring, say, three years with Java *or* Node.js will still report the candidate as needing familiarity with both, only at a lower level than a literal reading of "proficient" would imply.

5 Conclusions and Future Work

This work proposed a hybrid skill-gap analysis and course-recommendation framework that avoids two common failure modes: the semantic blindness of keyword matching and the instability of an end-to-end LLM. It makes four contributions. **(1)** A prompt-constrained extraction engine (Qwen3-Coder-480B, temperature 0) that reaches F1 96.7% against a keyword baseline's 84.4%, and is the only method able to resolve disjunctive requirements and proficiency levels – justifying the one place the system relies on a model. **(2)** A quantitative, progressive skill-gap model that reports not only which skills are missing but how many levels separate the candidate from the target, without generating phantom gaps from disjunctive or nested requirements. **(3)** Deterministic tree-based retrieval and ranking that outperform their LLM-driven counterparts: 99.2% vs. 63.3% recall at $\approx$25$\times$ lower latency with zero model calls (P4), and 75.4% vs. 73.8% ranking precision with perfect run-to-run reproducibility (P5). **(4)** An empirical account of *where* the LLM belongs, including two counter-intuitive findings: the 99.8% token reduction comes from deterministic filtering rather than from the tree (P2), and the guardrail $E(c)$ is redundant once upstream filtering is in place (P3).

The system has clear limits. Each course carries a single subcategory label, so cross-disciplinary courses may be pruned from a branch where they would have been useful, and recommendation quality is bounded by the quality of the underlying skill tags. The catalogue is scraped at a single point in time and is not synchronized with source updates. Extraction still depends on a cloud-hosted model – a smaller exposure than an end-to-end design, since retrieval and ranking run locally, but not a removed one. A candidate with an extremely large gap can receive an exhaustive rather than a focused path. Finally, the evaluation itself is bounded by a synthetic test set and single-annotator labeling (Section 4.8).

Future work follows directly from these limits: replacing the hierarchical tree with a knowledge graph (e.g., Neo4j) to let one course connect to multiple do-

mains; a scheduled maintenance pipeline to keep the catalogue synchronized with its source; local deployment of the extraction model to remove cloud dependency and keep candidate data on-premises; and a background process that mines developer forums and repositories to keep the skill-alias map current automatically, rather than by hand ; and reintroducing a lightweight disjunctive (`ANY`) mechanism to correctly model "choose-one" requirements without the added complexity of full branch-scoring.

References

1. Hugh Van Deventer, Mark Mills, and August Evrard, *"From Interests to Insights: An LLM Approach to Course Recommendations Using Natural Language Queries,"*, arXiv, https://arxiv.org/abs/2412.19312, 2024.
2. Luis Gasco, Hermenegildo Fabregat, Laura García-Sardiña, Paula Estrella, Casimiro Pio Carrino, Daniel Deniz, Alvaro Rodrigo, and Rabih Zbib, *"Talent-CLEF at CLEF2026: Skill and Job Title Intelligence for Human Capital Management,"* in Proc. ECIR'26, pp. 251–258, 2026.
3. Ioannis - Aris Kostis, Dimitrios Sarafis, Konstantinos Karamitsios, Kon-stantinos Kotrotsios, Kalliopi Kravari, Costin Badica, and Periklis Chatzimi-sios, *"Towards an Integrated Retrieval System to Semantically Match CVs, Job Descriptions and Curricula,"* in Proc. PCI'22, pp. 151–157, 2022.
4. Aili Li, Yong Li, and Xiyu Gao, *"Personalized Learning Path Recommendation Based on Knowledge Graphs: A Survey,"* Electronics, vol. 15, 2026.
5. Joy Jia Yin Lim, Ye He, Jifan Yu, Xin Cong, Daniel Zhang-Li, Zhiyuan Liu, Huiqin Liu, Lei Hou, Juanzi Li, Bin Xu, *"Personalized Learning Path Planning through Goal-Driven Learner State Modeling, "* in Proc. WWW '26, pp. 6067–6078, 2026.
6. Boxuan Ma, Tianyuan Yang, and Baofeng Ren, *"A Survey on Explainable Course Recommendation Systems,"* in Proc. Distributed, Ambient and Pervasive Interactions. HCII'24, pp. 273–287, 2024.
7. Qingyuan Meng, Qiuyu Ma, Yingtao Liu, and Chengying Liao, *"Design and Implementation of a Personalized Learning Recommendation Platform for Computer Science Majors ,"* in Proc. AIFE'25, pp. 651–657, 2025.
8. Amir Hossein Nabizadeh, José Paulo Leal, Hamed N. Rafsanjani, and Rajiv Ratn Shah, *"Learning path personalization and recommendation methods: A survey of the state-of-the-art,"* Expert Systems with Applications, vol. 159, pp. 113596, 2020.
9. Afsana Nasrin, Lijun Qian, Pamela Obiomon, and Xishuang Dong, *"Enhancing Learning Path Recommendation via Multi-task Learning,"* arXiv, https://arxiv.org/abs/2507.05295, 2025.
10. Naoki Otani, Nikita Bhutani, and Estevam Hruschka, *"Natural Language Processing for Human Resources: A Survey,"* in Proc. NAACL-HLT'25, pp. 583–597, 2025.
11. Simon De Vos, Christopher Bockel-Rickermann, Jente Van Belle, and Wouter Verbeke, *"Predicting Employee Turnover: Scoping and Benchmarking the State-of-the-Art,"* Bus. Inf. Syst. Eng., vol 67, pp. 733–752, 2025.
12. Kaiwen Wei, Kejun he, Xiaomian Kang, Jie Zhang, Ymyang, Li Jin, Zhenyang Li, Jiang Zhong, Richard He Bai, and Junnan Zhu, *"From Past To Path: Masked History Learning for Next-Item Prediction in Generative Recommendation,"* in Proc. ACL'26, pp. 10421–10441, 2026.

13. Xiaonan Yang, Senlin Luo, Jinjie Zhou, and Limin Pan, *"Highly goal-oriented learning path recommendation with the forgetting curve,"*, Applied Soft Computing, vol. 196, pp. 114988, 2026.
14. Yinfeng Zhou, Jinjin Li, Hailong Yang, Qingyuan Xu, Taoli Yang, and Danlu Feng, *"Knowledge structures construction and learning paths recommendation based on formal contexts,"* Int. J. Mach. Learn. Cyber., vol 15, pp. 1605–1620, 2024.
15. Fengjun Zhu, Yiping Wen, and Qi Fu, *"A Survey on Learning Path Recommendation,"* in Proc. ChineseCSCW'21, pp. 577–589, 2022.
16. Guoqing Zhu, Naga Anjaneyulu Kopalle, Yongzhen Wang, Xiaozhong Liu, Kemi Jona, and Katy Börnerm *"Community-based data integration of course and job data in support of personalized career-education recommendations,"* in Proc. AIST'20, vol 57 (1), 2020.